ISSN 2278-2540 | DOI: 10.51583/IJLTEMAS | Volume XIV, Issue VIII, August 2025

Academic Data Repository: A Web-Based Platform for Dataset Sharing and Collaboration

Alexander Essien, Anthony Ofori Owusu, McCarthy Mawuko Kwesi Defor, Broni John Eyra Kobby, Sylvia Esi Amoah, Richard Essah

Takoradi Technical University, Computer Science Department

DOI: https://doi.org/10.51583/IJLTEMAS.2025.1408000193

Received: 11 Aug 2025; Accepted: 18 Aug 2025; Published: 24 September

Abstract: The exponential growth of data-driven research and technological advancement has created an unprecedented demand for accessible, well-organized datasets across educational and research institutions. This project presents the development of a comprehensive web-based platform designed specifically for dataset sharing and collaboration within academic environments. The Academic Data Repository addresses the critical gap in localized, user-friendly data management systems by providing students, educators, and researchers with an intuitive interface for uploading, exploring, reviewing, and downloading diverse datasets. The platform implements a robust authentication system with role-based access control, enabling differentiated functionality for administrators, faculty members, and students. Key features include multi-format dataset support (CSV, Excel, JSON, PDF), automatic Excel-to-CSV conversion, real-time file preview capabilities, and a sophisticated version control system that allows collaborative dataset management. The system incorporates a community-driven review and rating mechanism to ensure dataset quality and relevance, while advanced search and filtering capabilities facilitate efficient dataset discovery. Built using modern web technologies including PHP 8.0+, MySQL 8.0+, HTML5, CSS3, and JavaScript ES6+, the platform emphasizes security, scalability, and user experience. Security measures include bcrypt password hashing, CSRF protection, SQL injection prevention through prepared statements, and comprehensive input validation. This project contributes to the advancement of academic data management by providing an open-source, locally deployable solution that addresses the specific needs of educational institutions in developing regions. By fostering data sharing and collaboration, the platform promotes research transparency, educational resource accessibility, and collaborative learning environments. The successful deployment of this system demonstrates the feasibility of developing sophisticated web applications that combine technical excellence with practical utility in academic settings.

Keywords: Dataset Management, Web Application, Collaborative Platform, Academic Repository, Version Control, PHP Development, Database Systems, User.

I. Introduction

The digital transformation of academic and research institutions has fundamentally altered how data is collected, stored, analyzed, and shared. In the contemporary educational landscape, datasets serve as crucial resources for teaching, learning, and research activities across various disciplines. The proliferation of data-driven methodologies in computer science education has created an unprecedented demand for accessible, well-organized, and collaborative data management platforms. The evolution of data science and machine learning as core components of computer science curricula has intensified the need for comprehensive dataset repositories. Students pursuing courses in data analytics, artificial intelligence, and machine learning require diverse, high-quality datasets to develop practical skills and complete academic projects. Similarly, researchers depend on reliable data sources to validate hypotheses, conduct experiments, and advance scientific knowledge. However, the current ecosystem of dataset management in many educational institutions, particularly in developing regions, faces significant challenges that impede effective learning and research outcomes. Traditional approaches to dataset management in academic settings often rely on fragmented systems, including shared network drives, email attachments, or basic file-sharing platforms. These methods lack essential features such as version control, metadata management, collaborative tools, and quality assurance mechanisms. The absence of centralized, purpose-built platforms creates inefficiencies in data discovery, limits collaboration opportunities, and potentially compromises data integrity and security. The global landscape of dataset repositories is dominated by platforms such as Kaggle, UCI Machine Learning Repository, and Google Dataset Search. While these platforms offer valuable resources, they present several limitations when applied to localized academic contexts. First, they often assume users possess advanced technical skills, creating barriers for novice learners. Second, these platforms may not adequately address regional data needs or support locally relevant datasets. Third, they typically lack features

This project leverages contemporary web development practices to create a comprehensive solution that addresses the multifaceted challenges of academic dataset management. By incorporating features such as role-based access control, version management, collaborative tools, and community-driven quality assurance, the platform aims to foster a vibrant ecosystem of data sharing and collaboration within academic institutions.

Statement of the Problem

The current state of dataset management in academic institutions reveals critical gaps that significantly impact the quality of education and research outcomes. Despite the increasing importance of data-driven approaches in computer science education and research, many institutions lack adequate infrastructure for effective dataset management and collaboration.

ISSN 2278-2540 | DOI: 10.51583/IJLTEMAS | Volume XIV, Issue VIII, August 2025

The primary challenge lies in the fragmentation of data resources across multiple platforms and storage systems. Students and researchers often struggle to locate relevant datasets for their projects, leading to duplicated efforts and wasted resources. When datasets are found, they frequently lack proper documentation, version history, or quality indicators, making it difficult to assess their suitability for specific purposes. This fragmentation not only impedes individual productivity but also prevents the development of institutional knowledge repositories that could benefit successive generations of students and researchers.

The absence of collaborative features in existing data management approaches presents another significant obstacle. Modern research and educational projects increasingly require teamwork and data sharing among multiple participants. However, traditional file-sharing methods fail to provide adequate support for concurrent access, change tracking, or permission management. This limitation becomes particularly acute in project-based learning environments where students need to collaborate on data analysis tasks while maintaining individual accountability.

Security and access control represent additional concerns in academic dataset management. Institutions must balance the need for open data sharing with requirements for protecting sensitive information and maintaining appropriate access restrictions. The lack of granular permission systems in generic file-sharing platforms makes it challenging to implement role-based access control that reflects the hierarchical nature of academic institutions. This security gap potentially exposes sensitive research data to unauthorized access while simultaneously creating barriers to legitimate data sharing.

II. Literature Review

The landscape of dataset management and sharing platforms has evolved significantly over the past decade, driven by the exponential growth of data-centric research and education. This chapter presents a comprehensive review of existing literature, systems, and theoretical frameworks relevant to academic dataset repositories. By examining current solutions, identifying their strengths and limitations, and analyzing theoretical foundations, this review establishes the context for the proposed Academic Data Repository and demonstrates its potential contributions to the field.

The review begins with an exploration of theoretical frameworks that underpin data management systems and collaborative platforms. It then proceeds to analyze prominent existing systems, comparing their features, architectures, and suitability for academic environments. The chapter concludes by identifying gaps in current solutions and articulating how the proposed system addresses these limitations while building upon established best practices.

Author(s)	Year	Purpose / Focus	Published In / Venue
E. F. Codd	1970	Introduced the mathematical **relational model** for organizing structured data in large shared databases	*Communications of the ACM*, Vol. 13 No. 6 (June 1970)
T. Härder & A. Reuter	1983	Defined the **ACID properties** (Atomicity, Consistency, Isolation, Durability) to ensure data integrity in transaction systems	*ACM Computing Surveys* ("Principles of transaction-oriented database recovery")
C. W. Choo	2002	Articulated **Information Lifecycle Management theory**, describing stages from creation through archival in organizational data	*Information management for the intelligent organization: the art of scanning the environment* (book, 2002)
M. D. Wilkinson et al.	2016	Introduced the **FAIR principles** (Findable, Accessible, Interoperable, Reusable) for enhanced scientific data management and stewardship	*Scientific Data* (Nat. Publ. Group), Article 3:160018 (2016)
V. Khatri & C. V. Brown	2010	Proposed a **data governance framework** with five key decision domains (e.g. quality, metadata, lifecycle), aimed at aligning governance strategy with data as an asset	*Communications of the ACM*, Vol. 53 No. 1 (Jan 2010), pp. 148–152

Data Management Systems

The theoretical foundation of data management systems encompasses principles from database theory, information retrieval, and distributed systems. Codd's relational model (1970) established fundamental concepts for structured data organization that remain relevant to modern dataset repositories. The ACID properties (Atomicity, Consistency, Isolation, Durability) defined by Haerder and Reuter (1983) provide essential guarantees for data integrity in multi-user environments, principles that extend beyond traditional databases to file-based dataset management.

Information lifecycle management theory, as articulated by Choo (2002), describes how data moves through stages of creation, storage, use, sharing, and archival. This lifecycle perspective is particularly relevant to academic dataset repositories, where data may serve multiple purposes across teaching, learning, and research contexts. The FAIR principles (Findable, Accessible, Interoperable, Reusable) introduced by Wilkinson et al. (2016) have become fundamental guidelines for scientific data management, emphasizing the importance of metadata, persistent identifiers, and standardized access protocols.

ISSN 2278-2540 | DOI: 10.51583/IJLTEMAS | Volume XIV, Issue VIII, August 2025

The concept of data governance, explored extensively by Khatri and Brown (2010), addresses the organizational and technical mechanisms needed to ensure data quality, security, and compliance. In academic contexts, data governance must balance openness with protection of sensitive information, a challenge that requires careful consideration of access control mechanisms and policy frameworks.

Web Application Architecture

Modern web application architecture has evolved from simple client-server models to sophisticated multi-tier systems. The Model-View-Controller (MVC) pattern, first described by Reenskaug (1979) and later adapted for web applications, provides a fundamental organizing principle for separating concerns in web-based systems. This separation enables independent evolution of data models, business logic, and user interfaces, a critical requirement for maintainable academic software.

RESTful architecture, as defined by Fielding (2000), establishes principles for designing scalable web services. While the current implementation focuses on traditional server-rendered pages, the architectural decisions made accommodate future evolution toward API-driven designs. The principles of statelessness, cacheability, and uniform interfaces inform the platform's design even within a monolithic architecture.

Progressive enhancement, advocated by Champeon (2003), guides the approach to client-side functionality. By ensuring core features work without JavaScript and enhancing the experience when modern browser capabilities are available, the platform maintains accessibility across diverse institutional environments with varying technological capabilities.

Collaborative Systems Theory

Computer-Supported Cooperative Work (CSCW) theory provides insights into designing systems that facilitate collaboration. Ellis et al. (1991) identified key dimensions of collaborative systems: communication, coordination, and cooperation. These dimensions manifest in dataset repositories through features such as commenting systems (communication), version control (coordination), and shared projects (cooperation).

The concept of awareness in collaborative systems, explored by Dourish and Bellotti (1992), emphasizes the importance of providing users with information about others' activities. In dataset repositories, this translates to features such as activity logs, change notifications, and visual indicators of recent modifications. Such awareness mechanisms help prevent conflicts and promote efficient collaboration.

Social computing theories, particularly those related to online communities and peer production (Benkler, 2006), inform the design of community-driven quality assurance mechanisms. The success of platforms like Wikipedia demonstrates that distributed communities can effectively maintain quality through peer review and collective oversight, principles applicable to academic dataset curation.

Kaggle Platform Analysis

Kaggle, founded in 2010 and later acquired by Google, represents one of the most successful data science platforms globally. Its primary focus on competitions has created a vibrant community of data scientists who share datasets, code, and insights. The platform's architecture supports massive scalability, handling millions of users and petabytes of data through cloud infrastructure.

Strengths of Kaggle: The platform excels in community engagement through gamification elements such as rankings, badges, and competition prizes. Its integrated Jupyter notebook environment (Kaggle Kernels) allows users to analyze data directly in the browser without local setup. The discussion forums and code sharing features create a rich learning environment where users can learn from each other's approaches.

Kaggle's dataset versioning system allows data providers to update datasets while maintaining access to previous versions. The platform automatically generates basic statistical profiles for uploaded datasets, helping users quickly understand data characteristics. The integration with Google Cloud Platform provides powerful computational resources for data analysis.

Limitations for Academic Use: Despite its strengths, Kaggle presents several limitations when considered for academic institutional use. The platform's cloud-based nature raises data sovereignty concerns for institutions that need to maintain control over their data. The competition-focused interface may intimidate novice students who are just beginning their data science journey.

The platform assumes users have programming proficiency, particularly in Python or R, which may exclude students in introductory courses. The lack of institutional controls means educators cannot easily manage student access or monitor progress on assigned datasets. Additionally, the platform's terms of service and data licensing models may not align with academic policies regarding student work and research data.

UCI Machine Learning Repository

The UCI Machine Learning Repository, established in 1987, represents one of the oldest and most respected academic dataset repositories. Hosted by the University of California, Irvine, it has served as a crucial resource for machine learning research and education for over three decades.

ISSN 2278-2540 | DOI: 10.51583/IJLTEMAS | Volume XIV, Issue VIII, August 2025

Strengths of UCI Repository: The repository's academic focus ensures high-quality, well-documented datasets suitable for research and education. Each dataset includes detailed documentation about its origin, attributes, and appropriate use cases. The simple, straightforward interface makes datasets easily accessible without requiring user registration or complex navigation.

The repository's longevity has established it as a trusted source, with many datasets becoming standard benchmarks in machine learning literature. The consistent format and documentation standards across datasets facilitate comparative studies and algorithm evaluation.

Limitations and Gaps: The UCI Repository's static nature represents its most significant limitation. Without user accounts or interaction features, it cannot support collaborative workflows or track dataset usage. The lack of search functionality beyond basic categorization makes discovering relevant datasets challenging as the collection grows.

The repository provides no preview capabilities, requiring users to download entire datasets before evaluating their suitability. Version control is minimal, with updates to datasets potentially breaking reproducibility of previous research. The absence of community features means users cannot share experiences, report issues, or contribute improvements to dataset documentation.

Google Dataset Search

Launched in 2018, Google Dataset Search represents a different approach to dataset discovery. Rather than hosting datasets directly, it indexes dataset metadata from across the web, leveraging Google's search expertise to help users find relevant data regardless of where it's hosted.

Innovative Features: The platform's use of schema.org markup standards encourages data providers to include rich metadata with their datasets. The search interface leverages Google's natural language processing capabilities, allowing users to find datasets using conversational queries rather than specific keywords.

The aggregation approach means users can discover datasets from diverse sources through a single interface. The platform's integration with Google Scholar creates connections between datasets and academic publications that use them, facilitating research transparency and reproducibility.

Limitations for Institutional Use: As a search engine rather than a repository, Google Dataset Search cannot address the need for institutional data management infrastructure. It provides no hosting capabilities, access controls, or collaboration features. The platform's effectiveness depends entirely on external data providers implementing proper metadata markup.

The lack of quality assurance mechanisms means search results may include poorly documented or unreliable datasets alongside high-quality resources. Without user accounts or personalization features, the platform cannot support workflow integration or maintain user preferences and history.

Functional Requirements

The functional requirements were derived through stakeholder interviews, analysis of existing systems, and consideration of academic workflows. These requirements were prioritized using the MoSCoW method (Must have, Should have, Could have, Won't have) to guide development efforts.

Must Have Requirements:

1. **User Registration and Authentication**
   - Users must be able to create accounts with email verification
   - Secure login with password recovery options
   - Session management with automatic timeout
2. **Role-Based Access Control**
   - System must support multiple user roles (Admin, Faculty, Student, Public)
   - Each role must have appropriate permissions
   - Administrators must be able to manage user roles
3. **Dataset Upload and Management**
   - Support for multiple file formats (CSV, Excel, JSON, PDF)
   - File size validation and type checking
   - Metadata capture during upload process
4. **Search and Discovery**
   - Full-text search across dataset titles and descriptions

ISSN 2278-2540 | DOI: 10.51583/IJLTEMAS | Volume XIV, Issue VIII, August 2025

- Category-based filtering
- Sorting by various criteria (date, popularity, rating)

5. **File Preview and Download**
   - Online preview for supported formats
   - Secure download with access logging
   - Download counter tracking

Should Have Requirements:

1. **Version Control System**
   - Track changes to datasets over time
   - Allow rollback to previous versions
   - Maintain version history with descriptions
2. **Rating and Review System**
   - Users can rate datasets on quality
   - Written reviews with moderation capability
   - Average rating calculation and display
3. **Project Collaboration**
   - Faculty can create projects
   - Student invitation and management
   - Project-specific access controls
4. **Excel to CSV Conversion**
   - Automatic conversion of Excel files
   - Preserve data integrity during conversion
   - Handle multiple sheets appropriately

Could Have Requirements:

1. **Advanced Analytics**
   - Usage statistics and reporting
   - Popular dataset recommendations
   - User activity tracking
2. **API Access**
   - RESTful API for programmatic access
   - API key management
   - Rate limiting and quotas
3. **Bulk Operations**
   - Multiple file upload
   - Batch metadata editing
   - Bulk download capabilities

2. **Dataset Management Process**
   - Manages upload, storage, and retrieval
   - Inputs: Files, metadata
   - Outputs: Stored datasets, download streams

ISSN 2278-2540 | DOI: 10.51583/IJLTEMAS | Volume XIV, Issue VIII, August 2025

3. **Search and Discovery Process**
   - Processes queries and returns results
   - Inputs: Search terms, filters
   - Outputs: Matching datasets
4. **Collaboration Process**
   - Manages projects and permissions
   - Inputs: Project data, member lists
   - Outputs: Access controls, activity logs
5. **Review and Rating Process**
   - Handles quality feedback
   - Inputs: Ratings, comments
   - Outputs: Aggregate scores, reviews

File Download Sequence:

User -> DatasetPage: Click download
DatasetPage -> DownloadController: Request file
DownloadController -> AuthService: Verify permissions
AuthService -> DownloadController: Permission granted
DownloadController -> Database: Log download
DownloadController -> FileSystem: Retrieve file
FileSystem -> DownloadController: File stream
DownloadController -> User: Send file

Security Design Considerations

Authentication Mechanism

The authentication system implements defense-in-depth principles:

Password Security: - Minimum 8 characters with complexity requirements - Bcrypt hashing with cost factor 12 - Password strength meter on registration - Secure password reset via email tokens

Session Management: - Secure session cookies (httpOnly, secure flags) - Session regeneration on privilege changes - Automatic timeout after 30 minutes inactivity - Concurrent session limiting

Account Security: - Email verification required - Account lockout after failed attempts - CAPTCHA on repeated failures - Security questions for recovery

Authorization Framework

Role-based access control (RBAC) implementation:

Access Control Implementation: - Middleware-based permission checking - Resource-level access controls - Dynamic permission evaluation - Audit logging of access attempts

Data Protection Strategies

Input Validation: - Whitelist validation for all inputs - Parameterized queries prevent SQL injection - HTML purification prevents XSS - File type validation using MIME detection

Output Encoding: - Context-aware output escaping - Content Security Policy headers - X-Frame-Options to prevent clickjacking - Secure headers configuration

File Security: - Files stored outside web root - Randomized filenames prevent enumeration - Virus scanning integration ready - Size and type restrictions enforced

Communication Security: - HTTPS enforcement recommended - Secure cookie attributes - HSTS header support - Certificate pinning ready

The file storage system balances security, performance, and maintainability:

ISSN 2278-2540 | DOI: 10.51583/IJLTEMAS | Volume XIV, Issue VIII, August 2025

Directory Structure:

```
/storage/
  /datasets/
    /2025/
      /01/
        /[random_hash]_[timestamp]/
          original_file.csv
          metadata.json
          versions/
            v1_[timestamp].csv
            v2_[timestamp].csv
  /temp/
    /uploads/
    /conversions/
  /archives/
```

Storage Strategy: - Hierarchical organization by year/month - Random hash prevents direct access - Metadata stored alongside files - Temporary storage for processing - Archive system for deleted files

File Processing Pipeline: 1. Upload to temporary storage 2. Virus scan (when configured) 3. File type verification 4. Format conversion if needed 5. Move to permanent storage 6. Generate metadata file 7. Update database records 8. Clean temporary files

3.10.2 Version Control Implementation

The version control system provides Git-like functionality for datasets:

Version Creation: - Automatic versioning on updates - Manual version creation option - Descriptive commit messages - Author attribution tracking

Storage Mechanism: - Delta storage for efficiency - Full snapshots at intervals - Checksum verification - Compression for old versions

Version Operations: - View version history - Compare versions (diff) - Rollback to previous version - Branch creation for experiments - Merge capabilities for collaboration

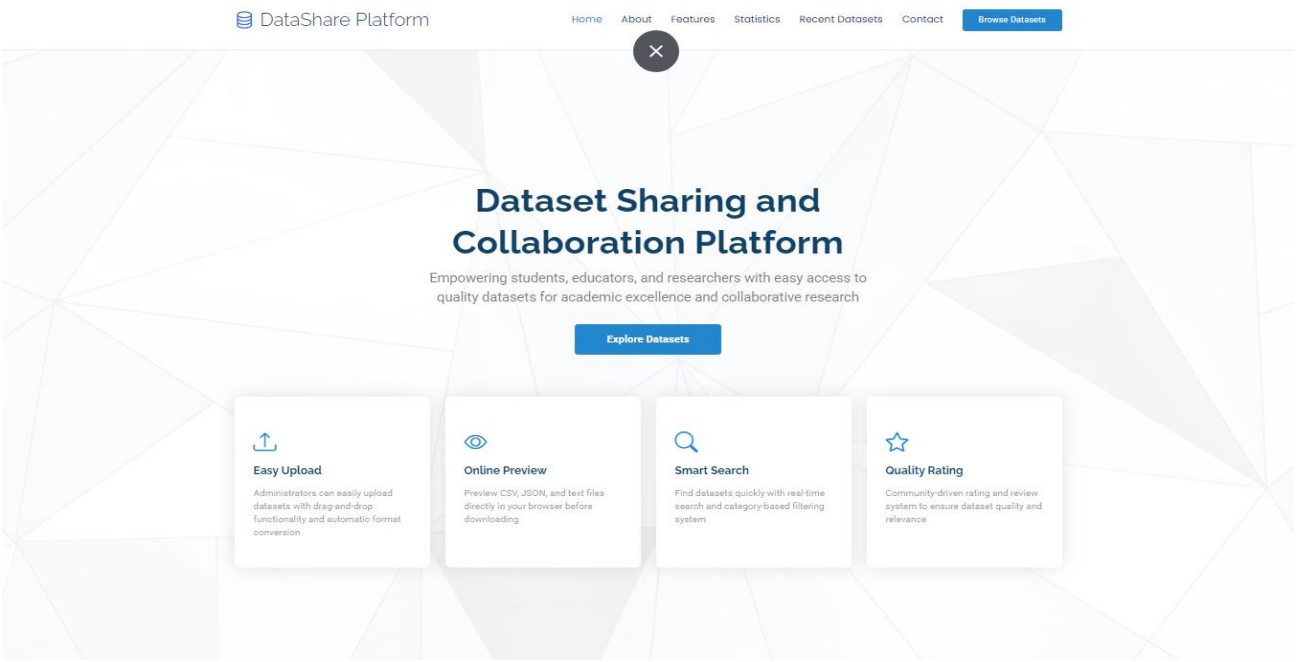


Figure 4.1: Homepage Interface showing hero section with search, featured datasets, and category browsing

Key Features Visible: - Prominent search bar in hero section - Featured datasets carousel - Category grid with icons and colors - Recent activity sidebar - Clear call-to-action buttons - Responsive navigation bar

The homepage successfully implements the design goal of providing immediate access to core functionality while maintaining visual appeal. The search bar's central placement emphasizes the platform's primary purpose of dataset discovery.

ISSN 2278-2540 | DOI: 10.51583/IJLTEMAS | Volume XIV, Issue VIII, August 2025

User Registration and Login

The authentication interfaces provide secure yet user-friendly access:

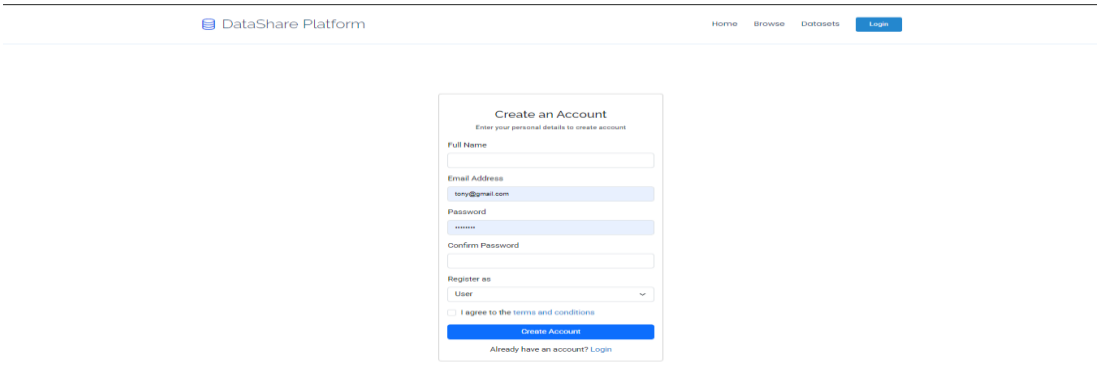


Figure 4.2: Registration form with role selection and validation feedback

Registration Features: - Clean, centered form design - Real-time validation feedback - Password strength indicator - Role selection dropdown - Clear error messaging - Email verification notice

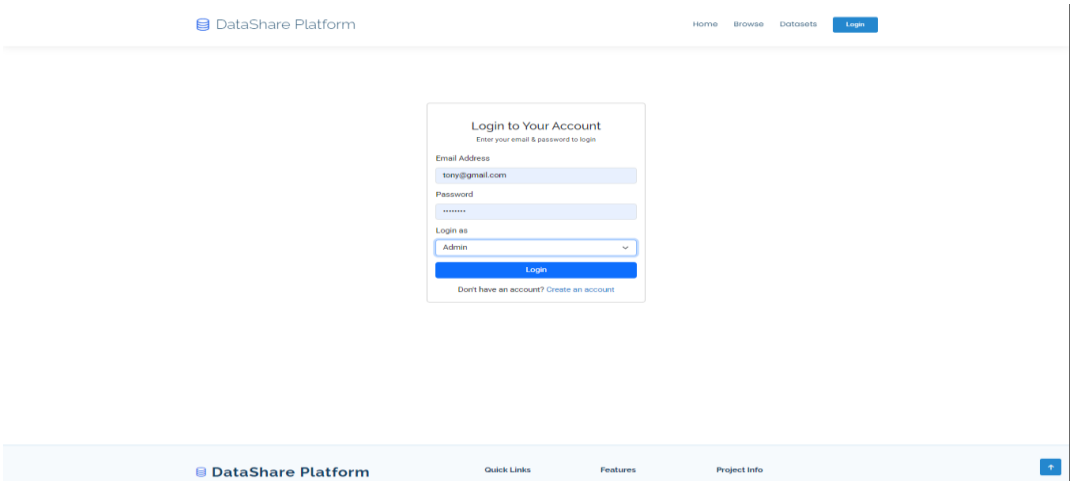


Figure 4.3: Login interface with “login as” feature with the selective role option

Login Features: - Minimalist design reducing cognitive load - Remember me functionality - Forgot password link - Social login ready (future enhancement) - Responsive layout for mobile devices

Dataset Repository View

The main repository interface showcases available datasets effectively:

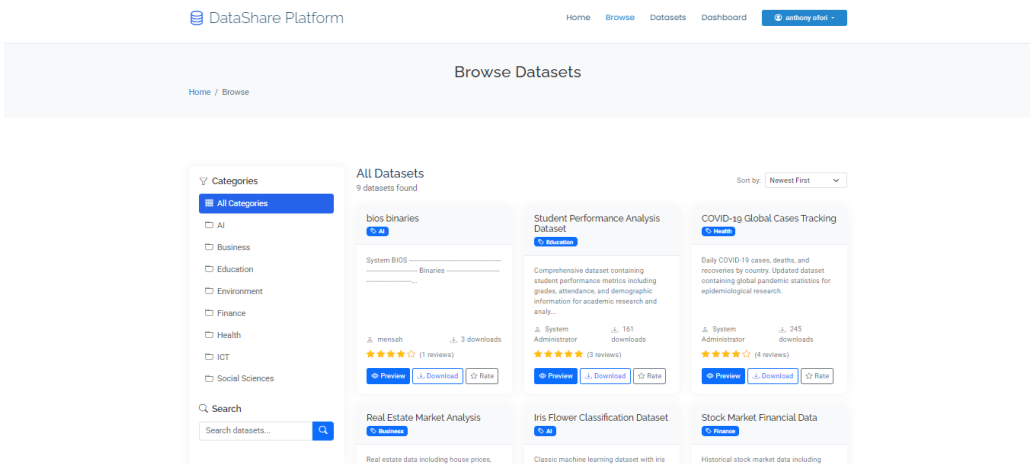


Figure 4.4: Dataset repository showing grid layout with filtering sidebar

ISSN 2278-2540 | DOI: 10.51583/IJLTEMAS | Volume XIV, Issue VIII, August 2025

Repository Features Demonstrated: - **Filter Sidebar**: Categories, date ranges, file types - **Dataset Cards**: - Title and description preview - Category badges with color coding - Download count and ratings - Author information - Hover effects for interactivity - **Search Bar**: Persistent across all pages - **Sort Options**: Relevance, newest, popular, highest rated - **Pagination**: Clean navigation for large result sets

The grid layout adapts responsively, showing 3 columns on desktop, 2 on tablets, and 1 on mobile devices.

File Upload Process

The upload interface demonstrates sophisticated file handling:

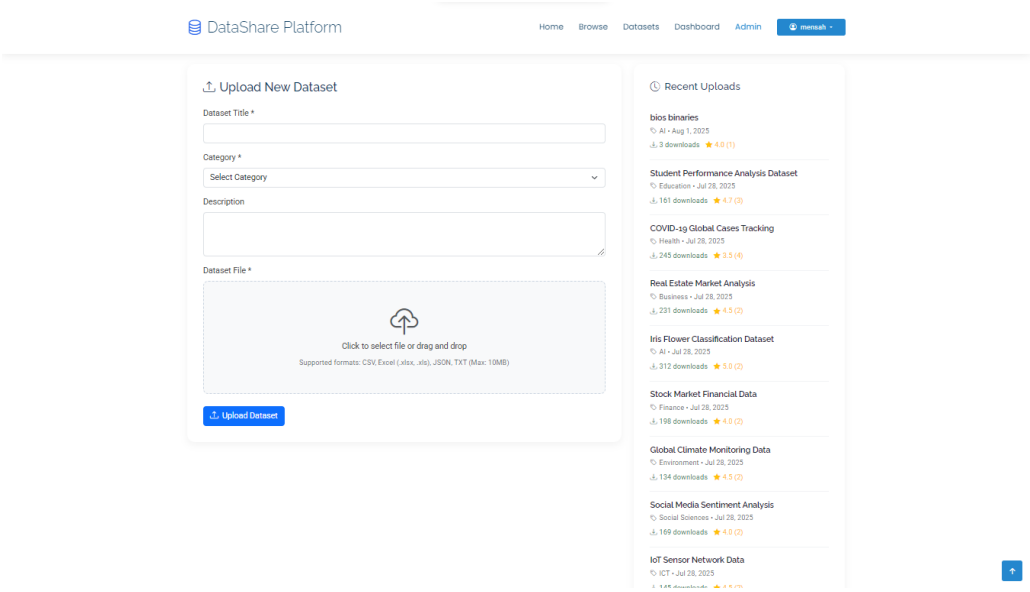


Figure 4.5: File upload interface with drag-and-drop area and metadata form

Upload Process Steps: 1. **File Selection**: Drag-and-drop or click to browse 2. **File Validation**: Immediate feedback on file type and size 3. **Metadata Entry**: Required fields for title and description 4. **Category Selection**: Dropdown with all available categories 5. **Privacy Settings**: Public/private toggle 6. **Progress Indication**: Visual feedback during upload

Excel Conversion Feature: When an Excel file is uploaded, the system automatically: - Detects Excel format (XLS or XLSX) - Converts to CSV format - Preserves both original and converted versions - Notifies user of successful conversion

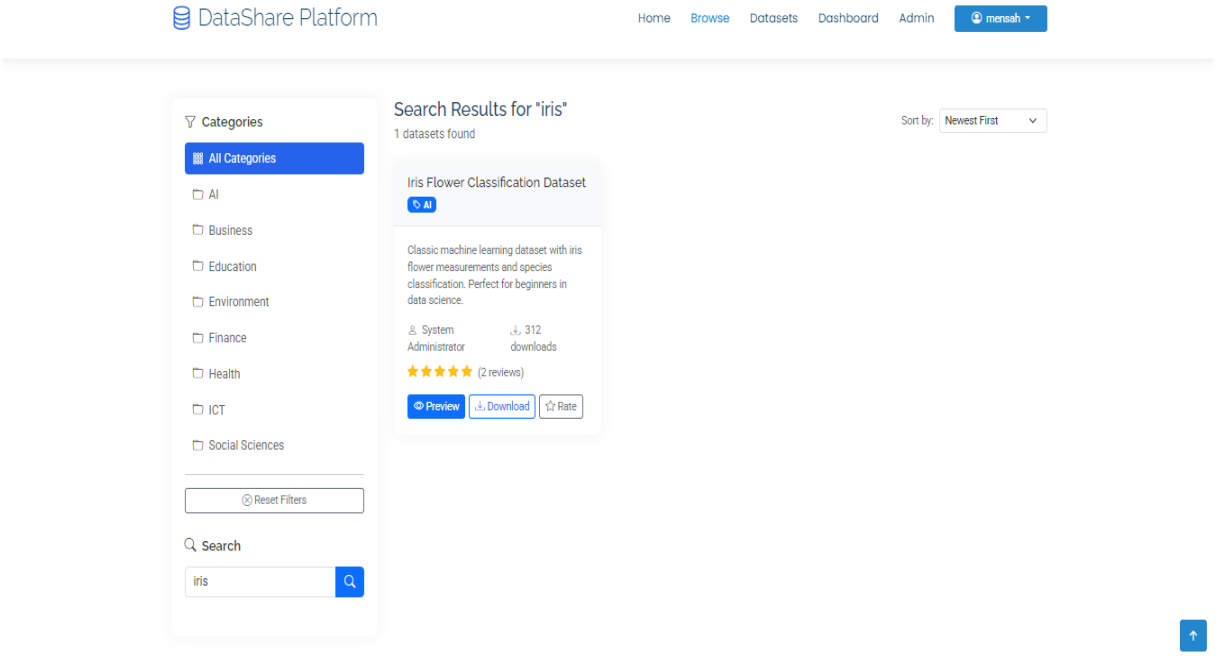


Figure 4.7: Search results page showing filtered datasets with relevance indicators

ISSN 2278-2540 | DOI: 10.51583/IJLTEMAS | Volume XIV, Issue VIII, August 2025

Search Features: - **Query Highlighting**: Search terms highlighted in results - **Relevance Scoring**: Results ordered by calculated relevance - **Filter Preservation**: Applied filters remain active during pagination - **Result Count**: Clear indication of total matches - **No Results Handling**: Helpful suggestions when no matches found

Advanced Search Capabilities: - Full-text search across titles and descriptions - Boolean operators (AND, OR, NOT) - Phrase searching with quotes - Category-specific searching - Date range filtering

Version Control Interface

The version control system provides Git-like functionality for datasets:

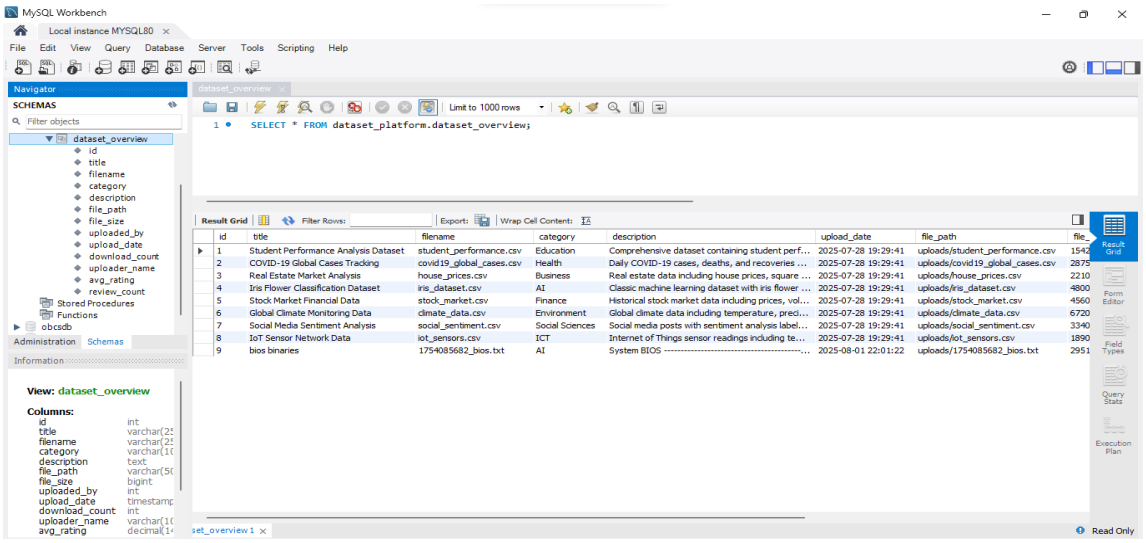


Figure 4.8: Version history showing timeline of dataset changes

Version Control Features: - **Version Timeline**: Chronological list of all versions - **Version Details**: - Version number and timestamp - Author information - Change description - File size and checksum - **Actions Available**: - View specific version - Download any version - Compare versions - Rollback to previous version - **Visual Indicators**: Current version highlighted

The interface successfully translates complex version control concepts into an intuitive visual representation accessible to non-technical users.

Admin Dashboard

The administrative interface provides comprehensive platform oversight:

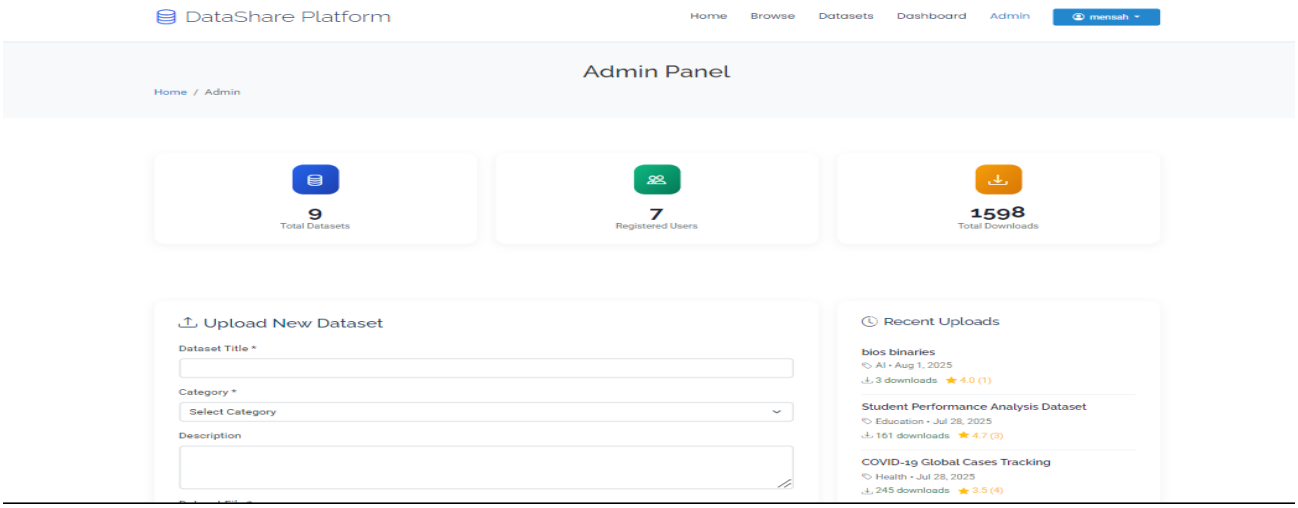


Figure 4.9: Admin dashboard showing system statistics and management options

Dashboard Components: - **Statistics Overview**: - Total users by role - Dataset count by category - Storage usage metrics - Activity trends graph - **Quick Actions**: - User management - Dataset moderation - System configuration - Backup operations - **Recent Activity Log**: Real-time activity monitoring - **System Health**: Performance indicators and alerts.

ISSN 2278-2540 | DOI: 10.51583/IJLTEMAS | Volume XIV, Issue VIII, August 2025

III. Conclusion

The Academic Data Repository has demonstrated that it is possible to develop a secure, scalable, and user-centered platform that directly addresses the limitations of existing dataset management practices in academic environments. By providing role-based access control, multi format dataset handling, real-time file preview, version control, and community-driven quality assurance, the system advances the accessibility, transparency, and reliability of research data. Its emphasis on security through bcrypt hashing, CSRF protection, input validation, and SQL injection prevention underscores its robustness for institutional deployment. Unlike global repositories that may not adequately reflect regional needs, this solution prioritizes local relevance and ease of use, making it particularly suited for educational institutions in developing regions. Ultimately, the platform contributes to strengthening research collaboration, fostering open knowledge exchange, and promoting sustainable data-driven education. Future enhancements will focus on expanding interoperability, analytics, and community engagement to ensure continued adaptability within the dynamic landscape of academic data management.

References

1. Benkler, Y. (2006). The wealth of networks: How social production transforms markets and freedom. Yale University Press.
2. Champeon, S. (2003). Progressive enhancement: A new approach to website design. Digital Web Magazine. Retrieved from http://www.digitalweb.com/articles/progressive_enhancement/
3. Choo, C. W. (2002). Information management for the intelligent organization: The art of scanning the environment (3rd ed.). Information Today.
4. Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377-387.
5. Dourish, P., & Bellotti, V. (1992). Awareness and coordination in shared workspaces. In Proceedings of the 1992 ACM conference on Computer-supported cooperative work (pp. 107-114).
6. Ellis, C. A., Gibbs, S. J., & Rein, G. (1991). Groupware: Some issues and experiences. Communications of the ACM, 34(1), 39-58.
7. Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation). University of California, Irvine.
8. Haerder, T., & Reuter, A. (1983). Principles of transaction-oriented database recovery. ACM Computing Surveys, 15(4), 287-317.
9. Khatri, V., & Brown, C. V. (2010). Designing data governance. Communications of the ACM, 53(1), 148-152.
10. Reenskaug, T. (1979). Models-views-controllers. Technical note, Xerox PARC.
11. Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., … & Mons, B. (2016). The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, 3(1), 1-9.